# Documentación de la API del alumno (OpenAPI 3.1)

El backend de `aprueba/aprueba_student_web` publica su contrato en formato **OpenAPI 3.1**, la convención que fija la empresa en `Aprueba_Generador_API.docx` ("contrato OpenAPI 3.1 generado desde el código").

| Qué | Dónde |
|---|---|
| Documentación navegable (Swagger UI) | https://aprueba-api.vercel.app/api/docs (pide usuario y contraseña) |
| Especificación (JSON) | https://aprueba-api.vercel.app/api/v1/openapi.json (misma protección) |
| Archivo en el repositorio | `backend/src/docs/openapi.json` |

En la documentación navegable, **Authorize** recibe el `accessToken` de `POST /api/v1/auth/login` y permite probar los endpoints con "Try it out". El servidor "Este servidor" apunta al mismo dominio que sirve la documentación.

## Acceso

La API es **privada**: la consumen la web, la app Flutter y la consola admin. Por eso su documentación no queda abierta en producción. El acceso lo controla la variable `API_DOCS` (`backend/src/middleware/docsAccess.js`):

| `API_DOCS` | Comportamiento |
|---|---|
| `off` | `404`, como si no existiera. **Valor por defecto en producción.** |
| `protected` | Usuario y contraseña (HTTP Basic) con `API_DOCS_USER` y `API_DOCS_PASSWORD`. Si falta alguna credencial, se cierra (`404`). |
| `public` | Abierta. Valor por defecto en desarrollo. |

En todos los casos se marca `noindex` para que no aparezca en buscadores.

La documentación **no da acceso a la API**: cada endpoint exige su propio token y, si corresponde, plan de pago. Lo que se protege es el mapa de la API, por defensa en profundidad.

## Cómo se genera

`openapi.json` **no se edita a mano**. Lo produce `backend/src/docs/generar.js` combinando tres fuentes, en este orden de prioridad:

1. **El código**:
   - análisis estático de `src/routes/*.js`: rutas, qué exige sesión o plan de pago, parámetros de ruta y de query, campos del body, estados de éxito y cada error declarado con `fail(res, …)`;
   - **respuestas reales**: corre el smoke test (118 pruebas) más un recorrido complementario, captura cada respuesta y de ahí infiere los esquemas y los ejemplos de respuesta y de body.
2. **El documento de la empresa** (`src/docs/contrato-api.json`): las fichas de `Aprueba_API_Backend.docx` aportan las descripciones, los tipos y la obligatoriedad de las entradas, y los errores documentados.

3. **Anotaciones** ( `src/docs/anotaciones.js` ): grupos, resúmenes y la documentación de los endpoints agregados después del documento ( `backend/ENDPOINTS_GAP.md` ).

Los ejemplos se limpian antes de publicarse: los tokens y contraseñas se reemplazan por marcadores.

## Comandos

Desde `aprueba/aprueba_student_web/backend` :

```
PAES_DIR=<carpeta con "PAES Chile *"> npm run openapi:generar
```

```
npm run openapi:verificar
```

`openapi:verificar` falla si:

- la especificación no es OpenAPI 3.1 válida;
- una ruta de Express no está documentada, o una documentada ya no existe;
- una operación no tiene resumen, grupo o respuesta de éxito con esquema;
- una operación que exige sesión no documenta el `401` .

**Al agregar o cambiar un endpoint, hay que regenerar y verificar antes del PR.** Si no, `openapi:verificar` lo detecta.

## Diferencias entre el documento y el código

El generador incluye lo que declara `Aprueba_API_Backend.docx` aunque el código no lo implemente, pero **lo marca** en la especificación con la nota *"Definido en Aprueba_API_Backend.docx; la implementación actual no lo usa"*, para no prometer comportamientos que no ocurren. Al generar el contrato aparecieron 16 diferencias:

| Tipo | Endpoint | Lo que el documento define y el código no implementa |
|---|---|---|
| Límite de intentos | `POST /auth/login` | `429 TOO_MANY_ATTEMPTS` |
| Límite de intentos | `POST /auth/password/forgot` | `429 RATE_LIMITED` |
| Límite de intentos | `GET /me/data-export` | `429 EXPORT_RATE_LIMITED` |
| Sesión | `POST /auth/refresh` | `401 REFRESH_TOKEN_EXPIRED` (el código usa `REFRESH_TOKEN_INVALID` para todo) |
| Sesión | `POST /auth/logout` | Recibir `refreshToken` y revocarlo |
| Login social | `POST /auth/social` | `409 EMAIL_LINKED_OTHER_PROVIDER` |
| Moderación | `POST /posts` | `422 CONTENT_BLOCKED` |
| Validación | `PATCH /me` | `400 VALIDATION_ERROR` |
| Validación | `POST /checkout/sessions` | `400 VALIDATION_ERROR` (el código responde `INVALID_PLAN` ) |
| Validación | `POST /me/subscription/change` | `400 VALIDATION_ERROR` (el código responde `INVALID_PLAN` ) |
| Pagos | `POST /checkout/sessions` | `502 PAYMENT_PROVIDER_ERROR` |

| Tipo | Endpoint | Lo que el documento define y el código no implementa |
|---|---|---|
| Pagos | `POST /webhooks/stripe` | Body `type` y `data`; `400 WEBHOOK_SIGNATURE_INVALID` y `WEBHOOK_MALFORMED` (el webhook está deshabilitado) |
| Práctica | `GET /practice/next` | Query `sessionId` |

Varias coinciden con pendientes ya registrados en `docs/seguridad-pendiente.md` (límites de intentos, revocación del refresh token, pagos). Las demás conviene revisarlas con la empresa: o se implementan, o se ajusta el documento.